

Task 4: Secure Login System

Objective:

Develop a secure login web application that allows users to register, log in, and manage sessions securely.

Features Required

1. User Registration & Login

- Create new user accounts.
- Store passwords securely using:
 - `bcrypt` (recommended)
 - or `Argon2`

2. Input Validation

- Validate username, email, and password fields.
- Prevent invalid inputs and malicious data.

3. SQL Injection Protection

- Use parameterized queries.
- Never concatenate user input directly into SQL queries.

4. Session Management

- Maintain user sessions after login.
- Provide a logout feature that destroys the session.

5. Optional: Two-Factor Authentication (2FA)

- OTP verification via email or authenticator app.
-

Recommended Tech Stack

Backend

- Python Flask

Database

- SQLite

Security

- bcrypt

Frontend

- HTML, CSS, Bootstrap
-

Project Structure

```
secure-login-system/
|
├── app.py
├── users.db
├── requirements.txt
|
├── templates/
│   ├── register.html
│   ├── login.html
│   └── dashboard.html
├── static/
│   └── style.css
```

Install Dependencies

```
pip install flask flask-bcrypt
```

Main Application (app.py)

```
from flask import Flask, render_template, request, redirect, session
from flask_bcrypt import Bcrypt
import sqlite3
```

```

app = Flask(__name__)
app.secret_key = "secret_key"

bcrypt = Bcrypt(app)

# Create database
def init_db():
    conn = sqlite3.connect('users.db')
    cur = conn.cursor()

    cur.execute("""
    CREATE TABLE IF NOT EXISTS users(
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        username TEXT UNIQUE,
        password TEXT
    )
    """)

    conn.commit()
    conn.close()

init_db()

@app.route('/')
def home():
    return redirect('/login')

@app.route('/register', methods=['GET','POST'])
def register():
    if request.method == 'POST':
        username = request.form['username']
        password = request.form['password']

        hashed = bcrypt.generate_password_hash(password).decode('utf-8')

        conn = sqlite3.connect('users.db')
        cur = conn.cursor()

        try:
            cur.execute(
                "INSERT INTO users(username,password) VALUES (?,?)",
                (username, hashed)
            )

```

```

        conn.commit()
    except:
        return "User already exists"

    conn.close()

    return redirect('/login')

return render_template('register.html')

@app.route('/login', methods=['GET','POST'])
def login():
    if request.method == 'POST':

        username = request.form['username']
        password = request.form['password']

        conn = sqlite3.connect('users.db')
        cur = conn.cursor()

        cur.execute(
            "SELECT password FROM users WHERE username=?",
            (username,)
        )

        user = cur.fetchone()
        conn.close()

        if user and bcrypt.check_password_hash(user[0], password):
            session['user'] = username
            return redirect('/dashboard')

        return "Invalid Credentials"

    return render_template('login.html')

@app.route('/dashboard')
def dashboard():
    if 'user' not in session:
        return redirect('/login')

    return render_template('dashboard.html',
        username=session['user'])

```

```
@app.route('/logout')
def logout():
    session.clear()
    return redirect('/login')

if __name__ == '__main__':
    app.run(debug=True)
```

register.html

```
<!DOCTYPE html>
<html>
<body>
<h2>Register</h2>

<form method="POST">
  <input type="text" name="username" placeholder="Username" required>
  <br><br>

  <input type="password" name="password" placeholder="Password" required>
  <br><br>

  <button type="submit">Register</button>
</form>

<a href="/login">Login</a>

</body>
</html>
```

login.html

```
<!DOCTYPE html>
<html>
<body>

<h2>Login</h2>

<form method="POST">
  <input type="text" name="username" placeholder="Username" required>
  <br><br>
```

```
<input type="password" name="password" placeholder="Password" required>
<br><br>

<button type="submit">Login</button>
</form>

<a href="/register">Register</a>

</body>
</html>
```

dashboard.html

```
<!DOCTYPE html>
<html>
<body>

<h2>Welcome {{ username }}</h2>

<a href="/logout">Logout</a>

</body>
</html>
```

Expected Output

Registration Page

- User enters username and password.
- Password is hashed and stored in database.

Login Page

- User logs in with credentials.
- Session is created.

Dashboard

Welcome Dharani
Logout

Report for Submission

Title

Secure Login System Using Flask and bcrypt

Tools Used

- Python
- Flask
- SQLite
- bcrypt

Features Implemented

- User Registration
- Password Hashing
- Secure Login
- Session Management
- SQL Injection Protection
- Logout Functionality

Result

A secure login application that stores hashed passwords and protects user authentication using modern security practices.